

Revisiting TwinNet: A Reproducibility Study and Comparative Evaluation of Graph Neural Network Architectures

Basmala Taha

*Egypt-Japan University of
Science and Technology (E-JUST)*
Alexandria, Egypt
basmala.taha@ejust.edu.eg

Bassem Mokhtar

*Department of Electrical Engineering
Faculty of Engineering
Alexandria University*
Alexandria 21544, Egypt
bmokhtar@alexu.edu.eg

Abstract—Network digital twins built on graph neural networks, such as TwinNet, model a network as a heterogeneous graph of paths, links, and queues, using Gated Recurrent Unit (GRU) cells to estimate per-flow delay under complex queueing policies. We show that a full GRU cell is not necessary to exploit the hop ordering it relies on: we propose an order-aware selective-gating aggregator, inspired by Mamba’s input-dependent selection mechanism, that reinstates hop ordering within a lighter, input-dependent gate in place of GRU’s fixed-form update. Dropped into an otherwise unmodified TwinNet pipeline, this backbone outperforms GRU itself as well as three other established aggregators, GraphSAGE, a Graph Attention Network (GAT), and an order-agnostic selective-gating variant, all evaluated under identical training conditions so the backbone is the only variable. It does so with roughly 29% fewer parameters and lower latency than GRU. A depth ablation further shows that fewer message-passing rounds outperform more, contrary to common assumption. Together, these results indicate that hop ordering, not the GRU cell specifically, is what matters for network delay prediction, with direct implications for how reliably a digital twin can gate safe network changes.

Index Terms—network digital twin, graph neural networks, TwinNet, GraphSAGE, graph attention networks, selective state space models

I. INTRODUCTION

Modern networks rely on QoS mechanisms, e.g., priority scheduling, Weighted Fair Queueing, and Deficit Round Robin, to meet SLAs on delay, jitter, and packet loss, but classical queueing approximations struggle once topology, routing, and queueing policy interact in complex ways. Graph Neural Networks are a natural fit, since a network already looks like a graph of links, queues, and paths exchanging information to estimate delay. TwinNet [1] frames this as heterogeneous message passing over paths, links, and queues, using GRU cells to update each entity’s state from its neighbors. This choice generalizes well to unseen topologies, but it has never been tested against alternatives such as order-agnostic mean-pooling (GraphSAGE [6]), attention (GAT [7]), or the input-dependent selective updates behind Mamba [8]. Whether any of these suit network delay prediction is open, since per-path delay accumulates hop by hop and may depend on hop order, something GRU captures and mean-pooling cannot.

We propose an order-aware selective-gating aggregator that reinstates the hop ordering GRU relies on while replacing its fixed-form gates with a lighter, input-dependent gate borrowed from Mamba’s selection mechanism. Because Mamba’s own graph adaptation, Graph-Mamba [10], is built around long-range sequence linearization and is not architecturally compatible with TwinNet’s heterogeneous, three-entity message passing, we do not reproduce it; instead we adapt its core idea, input-dependent selection, to TwinNet’s existing per-hop update, which is the specific mechanism our experiments test. To isolate what drives its performance, we compare it against four other backbones under identical conditions, keeping TwinNet’s features, hidden-state sizes, rounds, readout, loss, and training procedure fixed: the original GRU baseline, a GraphSAGE-style mean aggregator, a GAT aggregator, and an order-agnostic selective-gating variant differing from our backbone only in whether it preserves message order. Our backbone outperforms every other backbone tested, including GRU itself, while using roughly 29% fewer parameters, and a depth ablation further shows fewer rounds outperform more. Together, these results suggest hop order matters for network delay prediction, but a full GRU cell is not the only, or best, way to exploit it.

This distinction has operational weight: in a digital twin gating which routing or scheduling changes are safe to deploy, prediction accuracy separates a routine optimization from an SLA violation, and a more accurate backbone, at no added cost, means fewer approved actions that degrade service.

Section II reviews related work; Section III details the pipeline and each backbone; Section IV reports results; Section V concludes.

II. RELATED WORK

A. GNN-Based Network Digital Twins

GNNs for network performance modeling were established by RouteNet [3] and extended by RouteNet-Fermi [2], which models the network as a heterogeneous graph of links, queues, and paths to capture circular dependencies between routing, queueing, and traffic. TwinNet [1], the direct basis of this

study, extends this to per-flow delay, jitter, and loss under arbitrary QoS-aware queueing policies, using GRU cells across three iterated steps: Link+Queue→Path, Path→Queue, and Queue→Link. Later work reuses the same backbone for 5G/B5G slicing [14] and slicing management [15], generally without revisiting GRU itself. Surveys [12], [13] cover GNNs in networking and digital twins more broadly.

B. Message-Passing Neural Networks

TwinNet instantiates the Message-Passing Neural Network framework [4], in which the update function, how a node folds aggregated neighbor messages into its state, is a free design choice. TwinNet uses a GRU cell [5], treating incoming messages as an ordered, masked sequence.

C. Alternative Aggregation Mechanisms

GraphSAGE [6] replaces recurrent updates with inductive, permutation-invariant mean-pooling, discarding neighbor order for simplicity. GAT [7] instead learns a softmax-normalized attention weight per neighbor, at the cost of more parameters and a less stable optimization landscape under short training budgets. Mamba [8] introduced selective state-space models, making an SSM’s discretization parameters functions of the current input rather than fixed as in S4 [9]; Graph-Mamba [10] adapts this by linearizing graphs into node sequences for long-range context (a concurrent, similarly-named architecture appears in [11]). As noted in Section I, this linearization is not compatible with TwinNet’s per-edge heterogeneous structure, so we implement a simplified selective-gating aggregator that preserves Mamba’s input-dependent selection within TwinNet’s existing skeleton — an adaptation of the underlying mechanism, not a reproduction of Graph-Mamba, and evaluated as such.

D. Gap in Prior Comparative Studies

Prior comparisons typically test one alternative against the recurrent default; e.g., [16] evaluates attention within RouteNet-style architectures alone. We instead evaluate four architecturally distinct families, recurrent, permutation-invariant, attention-based, and selective-gating, under one training procedure, isolating the backbone as the single variable. Concurrent work by Zacarias et al. [17] (2025) benchmarks GraphSAGE, ResGatedGCN, and GraphTransformer for digital twins on round-trip-time/packet-loss over real-world topologies, a different task and data regime from the per-flow delay estimation studied here, and does not examine message ordering, the question we target.

III. METHODOLOGY

A. Base Architecture

We build on TwinNet [1]: a network sample is a heterogeneous graph over paths, links, and queues, each initialized from its own raw features (traffic rate/packet size; capacity/scheduling policy; size/priority/weight), zero-padded to a hidden-state dimension shared across all backbones. Over t rounds, paths collect from links and queues, queues from

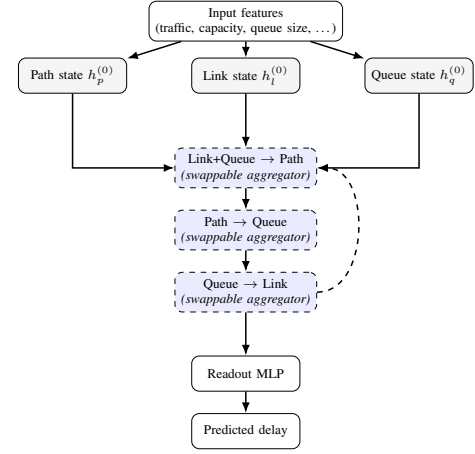


Fig. 1. TwinNet message-passing pipeline. Only the aggregator blocks (dashed) are replaced across backbones.

paths, and links from queues; final path states pass through a shared readout MLP (SELU activations, L_2 regularization) to predict per-path delay (Fig. 1). Only the update rule changes across backbones; everything else is identical, isolating it as the sole source of any performance difference.

B. GRU-Based Update (Baseline)

In TwinNet, each path folds the ordered sequence of link and queue messages along its route, padded and masked, into a state one hop at a time with a GRU cell [5]; Queue→Link works the same way. Path→Queue is the exception: incoming path messages are summed and passed through a single GRU call, with no notion of order. This asymmetry is preserved in our order-aware variant (Section III-D).

C. Permutation-Invariant and Attention-Based Aggregation

The GraphSAGE backbone averages neighbor states and combines the average with the node’s own previous state through one learned linear layer:

$$h_v^{(k)} = \sigma \left(W \cdot \left[h_v^{(k-1)} \parallel \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right] \right), \quad (1)$$

where $\parallel$ is concatenation and σ is SELU; it cannot distinguish message order. The GAT backbone learns a softmax-normalized attention weight per neighbor:

$$h_v^{(k)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W h_u^{(k-1)} + W h_v^{(k-1)} \right), \quad (2)$$

$$\alpha_{uv} = \text{softmax}_u \left(a^\top [W h_v \parallel W h_u] \right).$$

For Link+Queue→Path, link and queue messages are pooled before attention weights are computed, so the softmax normalizes jointly across both.

TABLE I

TEST-SET PERFORMANCE ACROSS MESSAGE-PASSING BACKBONES, SORTED BY MSE. LOWER IS BETTER FOR MSE/RMSE/MAE/LATENCY; HIGHER IS BETTER FOR R^2 .

Backbone	MSE	RMSE	MAE	R^2	Params	Lat.
Mamba, order-aware (V2)	0.132	0.364	0.208	0.909	16,067	$5.39\times$
GraphSAGE	0.184	0.429	0.274	0.874	6,691	$1.00\times$
GRU (baseline)	0.272	0.522	0.333	0.813	22,531	$7.62\times$
Mamba, order-agnostic (V1)	0.607	0.779	0.573	0.583	12,931	$1.16\times$
GAT	1.171	1.082	0.827	0.195	3,715	$1.60\times$

D. Selective-Gating Aggregation (Proposed)

We test two selective-gating variants, differing only in whether hop order is respected, to isolate the effect of ordering from the gating mechanism itself.

Order-agnostic (V1). Messages are mean-pooled as in Eq. (1) and folded in through an input-dependent gate rather than a fixed linear layer:

$$\begin{aligned} g_v^{(k)} &= \text{sigmoid}\left(W_g \left[h_v^{(k-1)} \parallel \bar{h}_{\mathcal{N}(v)}^{(k-1)} \right]\right), \\ h_v^{(k)} &= \sigma\left(g_v^{(k)} \odot W_c \left[h_v^{(k-1)} \parallel \bar{h}_{\mathcal{N}(v)}^{(k-1)} \right] + (1 - g_v^{(k)}) \odot h_v^{(k-1)}\right). \end{aligned} \quad (3)$$

Like GraphSAGE, this is permutation-invariant.

Order-aware (V2, proposed). We reinstate the padded, masked hop sequence used by GRU for Link+Queue→Path and Queue→Link, folding it step by step with a manually implemented scan whose gate is recomputed at every hop from that hop’s own input:

$$h_t = m_t \odot \left[g_t \odot c_t + (1 - g_t) \odot h_{t-1} \right] + (1 - m_t) \odot h_{t-1}, \quad (4)$$

where c_t, g_t come from $[h_{t-1} \parallel x_t]$ as in Eq. (3), x_t is the hop- t message, and m_t masks padding; the final state becomes the node’s new representation. Path→Queue keeps using Eq. (3) unchanged, mirroring the GRU baseline’s own order-agnostic treatment of that step.

E. Ablation Design

For the best-performing backbone, we additionally sweep message-passing depth $t \in \{2, 4, 8, 16\}$, fixing hidden-state size and all training hyperparameters, to test whether accuracy keeps improving with more rounds or eventually degrades.

IV. RESULTS AND DISCUSSION

A. Experimental Setup

All five backbones were trained under identical conditions: 20 epochs, 100 gradient steps/epoch, Adam (10^{-3}), MSE loss, hidden-state dimension $d = 32$ and depth $t = 8$ inherited from TwinNet’s original configuration. A seed was set once per notebook session rather than reset per run (Section IV-C discusses the resulting variance). MSE, RMSE, MAE, and R^2 are reported on a held-out test split of the NSFNet dataset.

B. Main Comparison

Table I summarizes test-set performance, parameter count, and relative inference latency.

Params/latency were measured by re-implementing all backbones under identical code and timing on a fixed synthetic batch on the same hardware; absolute latency is hardware-dependent, but relative ordering under matched conditions is informative. V2 uses roughly 29% fewer parameters than GRU (16,067 vs. 22,531) and is also faster.

The two selective-gating variants differ in exactly one respect, making their comparison the most informative in this table: mean-pooling before the gate (V1) discards hop order, as GraphSAGE does, and V1 was the second-weakest backbone. Reinstating the ordered, masked hop sequence, with no other change, is sufficient to make V2 the best-performing backbone overall. This also reframes GraphSAGE’s result: taken alone, GraphSAGE beating GRU could suggest order does not matter, but the gap between V2 and GraphSAGE (0.132 vs. 0.184 MSE) suggests order does matter, once paired with a lighter, more sample-efficient update than a full GRU cell.

A likely mechanism, not fully disentangled here, is that GRU learns one fixed gate applied uniformly across all path lengths and traffic conditions, whereas V2’s gate is recomputed per hop, letting a bottleneck link or queue be weighted more heavily than a routine hop. Since per-flow delay is often dominated by one or two bottleneck hops, an aggregator able to suppress low-information hops should isolate that signal more easily — consistent with the depth-ablation degradation at $t = 16$ (Section IV-C). A capacity-based explanation is also possible: V2’s smaller parameter count may simply be easier to fit within the 20-epoch budget used here, independent of any inductive-bias argument; separating these is future work. GAT remained weakest despite having the fewest parameters (Table I), so its result reflects an untuned learning rate/head count under a shared training budget, not a general claim against attention-based aggregation.

C. Ablation: Message-Passing Depth

Table II sweeps $t \in \{2, 4, 8, 16\}$ for V2, fixing all other hyperparameters. Contrary to the usual intuition, $t = 2$ was both most accurate and easiest to train (loss below 0.2 by epoch 18, vs. above 0.48 for $t = 16$ at epoch 20); $t = 16$ is the weakest of the four, roughly $2.8\times$ the MSE of $t = 2$, though it still generalizes reasonably ($R^2 = 0.776$). Two explanations seem plausible: NSFNet’s short paths may make extra rounds redundant, or a credit-assignment problem, each round lengthens the gradient’s path through learned gates, may compound optimization difficulty within a fixed epoch budget. Testing longer-path datasets is a natural next step. We also note a limitation: the $t = 8$ result here (0.196 MSE) does not match Table I’s $t = 8$ (0.132 MSE) despite identical settings, due to run-to-run variance from a seed set once per session rather than reset per run; averaging over several reset seeds would resolve this and is left as future work.

TABLE II
EFFECT OF MESSAGE-PASSING DEPTH ON THE ORDER-AWARE BACKBONE (V2).

t	MSE	RMSE	MAE	R^2
2	0.130	0.361	0.197	0.911
4	0.211	0.459	0.275	0.855
8	0.196	0.442	0.302	0.866
16	0.362	0.616	0.417	0.776

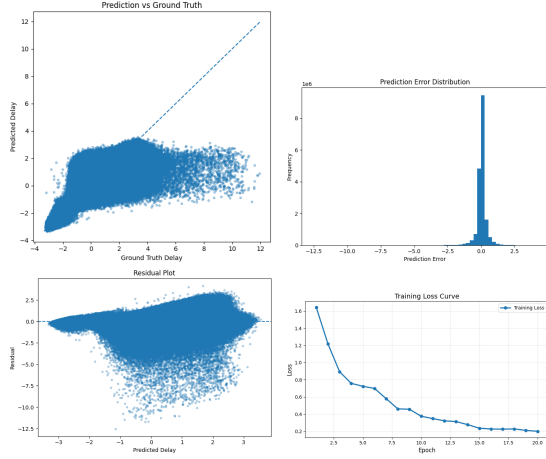


Fig. 2. Diagnostic plots for the order-aware backbone (V2): prediction, error, residual, and training loss.

D. Qualitative Diagnostics and Deployment Implications

Fig. 2 shows four diagnostic plots for V2, the best-performing model. The prediction-vs-truth scatter tracks the identity line closely for low-to-moderate delay, with mild flattening at the highest delays, consistent with its top R^2 score (Table I). The error distribution is sharply peaked near zero with only a modest tail, the residual plot stays tight and centered across most of the predicted-delay range, and the training loss decreases steadily and without instability across all 20 epochs, converging to the lowest final value of any backbone tested.

In a deployment setting, an operator would query the twin for each affected path’s predicted delay before approving a routing or scheduling change, approving only if every prediction stays below the SLA threshold. Every backbone under-predicts at the high end of the delay distribution, risking approval of SLA-violating actions; this effect is most severe for GAT and mildest for V2, which gives the operator the most trustworthy signal for exactly the borderline, high-delay cases that matter most.

V. CONCLUSION

We proposed an order-aware selective-gating aggregator for TwinNet-style network digital twins, a lighter alternative to a full GRU cell that still processes messages in hop order. Compared against GRU, GraphSAGE, GAT, and an order-agnostic selective-gating variant under identical training conditions, it outperformed every backbone tested while using roughly 29%

fewer parameters than GRU; a depth ablation showed fewer message-passing rounds outperform more. These accuracy differences directly affect how reliably a digital twin can validate routing or scheduling adjustments before deployment. Future work includes averaging over multiple random seeds to remove the run-to-run variance observed between Tables I and II, separate tuning for GAT, repeating the depth ablation on longer-path topologies, and quantifying the deployment scenario against a concrete SLA threshold.

REFERENCES

- [1] M. Ferriol-Galmés, J. Suárez-Varela, J. Paillisse, X. Shi, S. Xiao, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, “Building a digital twin for network optimization using graph neural networks,” *Computer Networks*, vol. 217, p. 109329, 2022.
- [2] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, “RouteNet-Fermi: Network modeling with graph neural networks,” *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, 2023.
- [3] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, “Unveiling the potential of graph neural networks for network modeling and optimization in SDN,” in *Proc. ACM Symp. SDN Research (SOSR)*, 2019, pp. 140–151.
- [4] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1263–1272.
- [5] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [6] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.
- [7] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Int. Conf. Learning Representations (ICLR)*, 2018.
- [8] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [9] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *Int. Conf. Learning Representations (ICLR)*, 2022.
- [10] C. Wang, O. Tsepa, J. Ma, and B. Wang, “Graph-Mamba: Towards long-range graph sequence modeling with selective state spaces,” *arXiv preprint arXiv:2402.00789*, 2024.
- [11] A. Behrouz and F. Hashemi, “Graph Mamba: Towards learning on graphs with state space models,” *arXiv preprint arXiv:2402.08678*, 2024.
- [12] J. Suárez-Varela, P. Almasan, M. Ferriol-Galmés, K. Rusek, F. Geyer, X. Cheng, X. Shi, S. Xiao, F. Scarselli, A. Cabellos-Aparicio, and P. Barlet-Ros, “Graph neural networks for communication networks: Context, use cases and opportunities,” *IEEE Network*, vol. 37, no. 3, pp. 146–153, 2022.
- [13] P. Almasan, M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, D. Perino, D. López, A. A. P. Perales, P. Harvey, L. Ciavaglia, L. Wong, V. Ram, S. Xiao, X. Shi, X. Cheng, and A. Cabellos-Aparicio, “Network digital twin: Context, enabling technologies, and opportunities,” *IEEE Communications Magazine*, vol. 60, no. 11, pp. 22–27, 2022.
- [14] R. Aben-Athar, L. Costa, J. Santos, A. Silva, L. Wanderley, C. Bastos, A. Klautau, and A. Frota, “Network digital twin for route optimization in 5G/B5G transport slicing with what-if analysis,” *arXiv preprint arXiv:2505.04879*, 2025.
- [15] H. Wang, Y. Wu, G. Min, and W. Miao, “A graph neural network-based digital twin for network slicing management,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 2, pp. 1367–1376, 2020.
- [16] B. K. Dhamala, B. R. Dawadi, P. Manzoni, and B. K. Acharya, “Performance evaluation of graph neural network-based RouteNet model with attention mechanism,” *Future Internet*, vol. 16, no. 4, p. 116, 2024.
- [17] I. Zacarias, O. Ben Taarit, and A. Jukan, “On Effectiveness of Graph Neural Network Architectures for Network Digital Twins (NDTs),” *arXiv preprint arXiv:2508.02373*, 2025.